

PRODUCT REQUIREMENTS DOCUMENT

Offline Facial Recognition & Liveness Detection System

for Datalake 3.0 — React Native Mobile Application

Hackathon Submission Document

Submission Window: 22 May 2026 – 05 June 2026

1. Document Information

Document Title	Product Requirements Document — Offline Facial Recognition & Liveness Detection
Version	1.0
Status	Draft — For Hackathon Submission
Prepared For	Hackathon Evaluation Committee
Target Application	Datalake 3.0 (React Native, Android & iOS)
Submission Deadline	05 June 2026
Audience	IT Students, AI/ML Enthusiasts, App Developers, Tech Professionals, Startups

2. Executive Summary

The Datalake 3.0 application serves field personnel operating in remote, zero-connectivity environments. Authenticating these personnel securely, without reliance on network infrastructure, is a critical unresolved gap. This Product Requirements Document defines the full specification for an offline-first, AI-powered facial recognition and liveness detection module to be embedded within the existing React Native application.

The solution must achieve greater than 95% facial recognition accuracy, complete end-to-end verification in under one second, remain within a 20 MB model footprint, and operate exclusively using open-source technologies. The module must also support deferred synchronisation with an AWS backend when connectivity is restored, with automatic local data purging post-sync.

3. Goals & Success Metrics

3.1 Primary Goals

- Authenticate field personnel reliably in zero-network conditions using on-device facial recognition.
- Prevent spoofing attacks (printed photographs, digital screens) via offline liveness detection.
- Maintain a lightweight, non-bloating footprint within the existing Datalake 3.0 React Native package.
- Ensure cross-platform consistency across Android 8.0+ and iOS 12+ on mid-range hardware.
- Enable an offline-to-online sync pipeline with automatic local purge post-synchronisation.

3.2 Non-Goals (Out of Scope)

- Cloud-based or real-time facial recognition during authentication.
- Integration with third-party commercial SDKs (licensed or paid).
- High-end device-exclusive features (e.g., 3D depth sensors, IR cameras).
- Cross-system identity federation beyond the Datalake 3.0 ecosystem.

3.3 Key Success Metrics

Metric	Target
Facial recognition accuracy (Top-1)	> 95% on diverse Indian demographics
End-to-end verification latency	< 1 second on a 3 GB RAM, mid-range device
AI model file size	≤ 20 MB (target: ≤ 12 MB)
Liveness detection false-accept rate	< 2% (live person misidentified as spoof)
Liveness detection false-reject rate	< 5% (live person rejected)
Offline session data sync success	100% after connectivity restoration
False positive authentication rate	< 0.1%
Supported OS coverage	Android 8.0+ and iOS 12+

4. Stakeholders & User Personas

4.1 Stakeholders

Role	Interest / Responsibility
Field Personnel	Primary end-users who authenticate via the app daily in remote areas
Field Operations Manager	Requires reliable attendance records with anti-fraud assurance
IT / DevOps Team	Owns Datalake 3.0 codebase; responsible for module integration and AWS sync pipeline
Security Officer	Ensures authentication mechanisms meet privacy and data handling standards
Hackathon Evaluators	Assess innovation, feasibility, scalability, and presentation quality

4.2 Primary User Persona

Rajan, 34, Field Survey Officer — Works in rural districts across India. His device is a Samsung Galaxy A-series (3 GB RAM, Snapdragon mid-range SoC). He operates in areas with no 4G coverage, often under harsh outdoor lighting conditions including direct sunlight and partial shade. He needs to punch in attendance reliably in under two seconds, without being blocked by network timeouts or authentication failures.

5. Functional Requirements

5.1 Facial Recognition Engine

1. The system shall enroll a user's facial embedding on-device during initial setup, storing it in encrypted local storage.
2. Recognition shall use a MobileNetV2 or MobileFaceNet backbone, quantized to INT8, enabling inference under 350 ms on the target hardware class.
3. The embedding vector shall be 128-dimensional, compared against stored embeddings using cosine similarity with a configurable threshold (default: 0.75).
4. The model shall be trained on a dataset inclusive of diverse Indian skin tones, face structures, and age demographics, validated across subgroups.
5. The system shall handle at least 500 enrolled facial profiles per device without degrading performance.
6. Re-enrollment shall be possible only through an authorised admin workflow.

5.2 Liveness Detection (Anti-Spoofing)

7. The system shall perform active liveness checks requiring at least one of the following gestures: blink, smile, or slight head turn.
8. A passive liveness check (texture/depth analysis using a Binary Cross-Entropy model on a single RGB frame) shall run in parallel to detect printed photographs and screen replays.
9. The liveness module shall process and respond within 400 ms.
10. The combined liveness pipeline (active + passive) shall complete before granting authentication approval.
11. Anti-spoofing shall function under outdoor lighting variations including direct sunlight, shadows, and low-light conditions.

5.3 Offline Operation

12. All recognition and liveness inference shall execute entirely on-device, with zero network calls during the authentication flow.
13. Authentication events shall be logged locally with a timestamp, user ID, verification outcome, liveness result, and GPS coordinates where available.
14. Local session logs shall be stored in encrypted SQLite or Realm database within the app sandbox.
15. The app shall function across unlimited offline sessions until storage is saturated.

5.4 Sync & Purge Mechanism

16. Upon detection of network connectivity, the app shall automatically queue pending authentication records for upload to the configured AWS endpoint.
17. Sync shall use exponential back-off retry logic (max 5 retries, delays: 2s, 4s, 8s, 16s, 32s) to handle intermittent connectivity.

18. Upon confirmed server acknowledgement (HTTP 200 with record IDs), the corresponding local records shall be purged.
19. A manual sync option shall also be accessible from the app settings screen.
20. Sync events shall themselves be logged (sync timestamp, record count, success/failure status).

5.5 React Native Integration

21. The facial recognition and liveness module shall be exposed as a single React Native Native Module (NativeModule) with a clean JavaScript API.
22. The JavaScript interface shall expose at minimum: enroll(userId, imagePath), verify(imagePath), livenessCheck(videoBufferOrFrames), syncPendingLogs().
23. The module shall be compatible with React Native 0.71 and above.
24. Camera access shall be managed through react-native-vision-camera or the standard CameraRoll API, with appropriate runtime permission prompts.
25. The module shall support both debug and release build configurations.

6. Non-Functional Requirements

6.1 Performance

- Total end-to-end latency (face detection → recognition → liveness result): < 1 second.
- Model cold-load time on first app launch: < 3 seconds on mid-range device.
- Memory footprint of inference engine at runtime: < 150 MB RAM.
- Battery impact: no persistent background processing; inference triggered only on user action.

6.2 Model Size & Compression

- Base model file size: ≤ 20 MB (target ≤ 12 MB).
- Quantization technique: Post-Training Quantization (PTQ) to INT8 using TensorFlow Lite or ONNX Runtime Mobile.
- Optional: Knowledge Distillation from a larger teacher model (e.g., ArcFace ResNet-50) to the student MobileFaceNet.
- Pruning: Structured filter pruning at ≥ 30% without accuracy drop below threshold.

6.3 Security & Privacy

- Facial embeddings shall never be stored as raw images; only the 128-d embedding vector shall be persisted.
- All local data (embeddings, session logs) shall be encrypted using AES-256 at rest.
- Sync transport to AWS shall use TLS 1.2 or higher.
- The solution shall comply with India's IT Act 2000 and applicable data localisation guidelines.
- No biometric data shall be transmitted to any third-party endpoint.

6.4 Reliability & Availability

- Authentication flow shall succeed or fail gracefully with a user-facing error message; no silent crashes.
- The app shall handle camera unavailability, insufficient lighting warnings, and obscured-face scenarios with explicit UI feedback.
- Crash-free session rate target: > 99.5%.

6.5 Compatibility

- Android: API Level 26 (Android 8.0) and above.
- iOS: iOS 12 and above.
- Minimum RAM: 3 GB.
- Architecture: ARM64 (primary), ARMv7 (fallback).
- TFLite runtime: bundled within the app, no device-level dependency.

7. System Architecture Overview

7.1 High-Level Component Breakdown

Component	Responsibility
React Native UI Layer	Camera preview, gesture prompts, authentication status screens, sync trigger UI
Native Module Bridge (JS ↔ Native)	Exposes enroll(), verify(), livenessCheck(), syncPendingLogs() APIs to JS
Face Detection Module	Detects and crops face ROI from camera frames using a lightweight MTCNN or MediaPipe Face Mesh
Recognition Inference Engine	Runs INT8-quantized MobileFaceNet TFLite model to generate 128-d embedding
Liveness Detection Engine	Active: blink/smile/head-turn via landmark delta; Passive: anti-spoof CNN on RGB frame
Embedding Store	Encrypted SQLite / Realm DB storing user ID → embedding vector mappings
Session Log Store	Encrypted local log of authentication events pending sync
AWS Sync Service	Background task to upload pending logs to AWS API Gateway → S3/DynamoDB on reconnect

7.2 Technology Stack

Layer	Technology
Mobile Framework	React Native 0.71+ (JavaScript + Native Modules)

Face Detection	MediaPipe Face Mesh (open-source, on-device)
Recognition Model	MobileFaceNet — TFLite INT8 quantized (open-source, Apache 2.0)
Liveness Detection	Custom lightweight CNN + MediaPipe Landmark delta tracking
Inference Runtime	TensorFlow Lite (Android) / Core ML via ONNX (iOS)
Local Database	SQLite via react-native-sqlite-storage (open-source, MIT)
Encryption	SQLCipher (open-source, BSD) for AES-256 at-rest encryption
Camera Access	react-native-vision-camera (open-source, MIT)
AWS Sync	AWS SDK (open-source) — API Gateway + S3/DynamoDB
Model Compression	TFLite Model Maker (post-training quantization), Netron for validation

7.3 Data Flow

26. User opens authentication screen; camera feed initialised.
27. MediaPipe detects face bounding box; image cropped and normalised to 112×112 px.
28. Active liveness prompt displayed (e.g., 'Please blink'); landmark delta computed over 30-frame buffer.
29. Passive anti-spoof CNN run on single frame in parallel.
30. On liveness pass: MobileFaceNet generates 128-d embedding.
31. Cosine similarity computed against stored embedding for matched user ID.
32. If similarity $\geq$ threshold: authentication granted; event logged locally.
33. On network restore: background sync service uploads pending logs, receives server ACK, purges local records.

8. AI Model Specification

8.1 Facial Recognition Model — MobileFaceNet

Parameter	Value
Base Architecture	MobileFaceNet (bottleneck residual blocks, depth-wise separable conv)
Input Resolution	112 × 112 × 3 (RGB)
Output Embedding	128-dimensional L2-normalised vector
Training Loss	ArcFace (Additive Angular Margin) with margin $m=0.5$, scale $s=64$
Training Dataset	MS-Celeb-1M (cleaned subset) + supplementary Indian demographics dataset
Pre-Quantization Size	~8 MB (FP32)

Post-Quantization Size	~2–3 MB (INT8)
Inference Latency	~80–120 ms on Snapdragon 665 (3 GB RAM device)
License	Apache 2.0 (open-source)

8.2 Liveness Detection Model

Parameter	Value
Passive Anti-Spoof	Lightweight CNN (3 conv blocks + global avg pool) trained on NUAA, CASIA-FASD datasets
Active Liveness	MediaPipe Face Mesh 468-landmark delta tracking across 30-frame temporal window
Input (passive)	Single RGB frame, 64 × 64 px
Output	Binary: real / spoof, confidence score
Model Size (passive)	< 1.5 MB (INT8 quantized)
Inference Latency	~40–60 ms passive; ~200 ms active gesture verification
Training Data License	NUAA (research-use, public); CASIA-FASD (public research)

8.3 Compression Strategy

- Stage 1 — Post-Training Quantization (PTQ): Convert FP32 weights to INT8 using representative calibration dataset. Expected 4× size reduction with < 1% accuracy drop.
- Stage 2 — Structured Pruning: Remove filters with L1-norm below threshold (30% sparsity target). Re-fine-tune for 5 epochs after pruning.
- Stage 3 — Knowledge Distillation (optional): Train MobileFaceNet student against ArcFace ResNet-50 teacher. Soft-label KL divergence loss. Improves accuracy-to-size ratio.
- Stage 4 — TFLite Flatbuffer serialization: Final model stored as .tflite for Android; ONNX → Core ML conversion pipeline for iOS.

9. Mandatory Deliverables

Deliverable	Description & Acceptance Criteria
Working Prototype (Android APK + iOS IPA)	Functional React Native app demonstrating enrollment, offline verification, liveness detection, and sync/purge on both platforms. Must run on a mid-range device without network.
Source Code Repository	Full source on GitHub/GitLab. MIT or Apache 2.0 licensed. Includes React Native project, native module (Android Java/Kotlin + iOS Swift/Obj-C), model training scripts, and quantization pipeline.
Offline Liveness Demo	Screen recording demonstrating blink/smile/head-turn active liveness and spoof rejection using a printed photograph test case.

Sync & Purge Demo	Screen recording or step-by-step document showing local logs uploading to AWS endpoint and purging upon ACK.
Technical Documentation	Architecture diagrams, model training pipeline, integration guide (step-by-step for Datalake 3.0 team), API reference, and benchmark tables.
Presentation Deck (.pptx / .pdf)	Max 20 slides. Covers problem, solution architecture, model benchmarks, integration plan, and live demo screenshots.

10. React Native Integration Guide (Summary)

10.1 Module Installation

The native module is distributed as an npm package. Install via: `npm install rn-offline-face-auth`. Run `npx pod-install` for iOS. Rebuild the project for both platforms.

10.2 JavaScript API Reference

Method	Signature & Returns
Enroll User	<code>enroll(userId: string, imagePath: string): Promise<{success: boolean, embedding: number[]}></code>
Verify User	<code>verify(userId: string, imagePath: string): Promise<{authenticated: boolean, confidence: number, latencyMs: number}></code>
Liveness Check	<code>livenessCheck(frameBuffer: Buffer[], gestureType: 'blink' 'smile' 'turn'): Promise<{isLive: boolean, confidence: number}></code>
Sync Pending Logs	<code>syncPendingLogs(endpoint: string, authToken: string): Promise<{synced: number, failed: number}></code>
Delete Enrollment	<code>deleteEnrollment(userId: string): Promise<{success: boolean}></code>

10.3 Required Permissions

- Android: CAMERA, WRITE_EXTERNAL_STORAGE (API < 29), ACCESS_FINE_LOCATION (for GPS tagging of auth events).
- iOS: NSCameraUsageDescription, NSLocationWhenInUseUsageDescription.

10.4 Model Asset Bundling

Place the following assets in `android/app/src/main/assets/` and `ios/YourApp/`: `face_recognition.tflite` (≤ 3 MB), `liveness_passive.tflite` (≤ 1.5 MB), `face_detection.tflite` (≤ 1 MB). For iOS, also include the Core ML `.mlmodel` variants. Total additional bundle size: ≤ 6 –8 MB.

11. Evaluation Criteria Alignment

Criterion (Marks)	How This PRD Addresses It
Innovation (30)	Specifies INT8 quantization + structured pruning + optional KD; passive + active hybrid liveness; MobileFaceNet as the edge backbone. Model target ≤ 12 MB well below 20 MB cap.
Feasibility (30)	Defines a clean React Native Native Module API; TFLite for Android + Core ML for iOS with documented integration steps; latency budget broken down per sub-component.
Scalability & Sustainability (20)	Specifies exponential back-off sync, server-ACK-gated purge, encrypted local storage, and per-user embedding management for up to 500 profiles.
Presentation & Documentation (20)	Mandates architecture diagram, API reference, benchmark tables, integration guide, and a concise slide deck as formal deliverables with defined acceptance criteria.

12. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Accuracy drop after quantization	Medium	High	Validate on representative Indian demographics holdout set; use QAT (Quantization-Aware Training) if PTQ drops accuracy below 95%
Liveness false-reject in bright sunlight	Medium	Medium	Adaptive thresholding based on ambient brightness sensor reading; UX guidance for user to shade face
iOS Core ML conversion issues	Low	High	Maintain ONNX as intermediate format; use coremltools 6.x; test on iOS 12, 14, and 17 simulators
React Native version incompatibility	Low	Medium	Build against RN 0.71 (LTS); document peer dependency ranges; provide polyfills for RN < 0.71
Sync data loss on app kill during upload	Medium	High	Write-ahead log approach: mark records 'in-flight' before upload; revert to 'pending' on non-ACK restart

13. Suggested Development Timeline

Phase	Duration	Key Activities
Phase 1: Model Research & Selection	3 days	Benchmark MobileFaceNet variants; identify Indian demographics dataset; finalise quantization pipeline
Phase 2: Model Training & Compression	5 days	Train / fine-tune MobileFaceNet; PTQ + pruning; validate accuracy on holdout; convert to TFLite + Core ML

Phase 3: Native Module Development	5 days	Android (Kotlin) + iOS (Swift) native wrappers; TFLite / Core ML inference integration; encrypted SQLite store
Phase 4: Liveness Detection Integration	3 days	MediaPipe landmark active liveness; passive anti-spoof CNN; combined pipeline testing
Phase 5: React Native Bridge & UI	3 days	JS API surface; camera integration; enrollment + verification flows; sync/purge UI
Phase 6: AWS Sync Pipeline	2 days	Background sync task; back-off retry; ACK-gated purge; manual sync trigger
Phase 7: Testing & Benchmarking	3 days	Performance profiling on mid-range device; accuracy testing; liveness spoof tests; integration tests
Phase 8: Documentation & Presentation	2 days	Technical docs, integration guide, benchmark tables, slide deck, source cleanup

14. Testing Phase

A structured, multi-layered testing strategy is mandatory before submission. Testing is organised across five phases: unit testing, component integration testing, system-level end-to-end testing, device compatibility testing, and security & privacy testing. Each phase has defined test cases, pass/fail criteria, and tooling.

14.1 Unit Testing

Unit tests validate each individual module in isolation before integration. All units must achieve $\geq 90\%$ code coverage.

Test ID	Module	Test Case	Pass Criteria
UT-01	Face Detection	Single face detected in 112×112 px crop from full-frame input	Bounding box IoU ≥ 0.85 vs ground truth
UT-02	Face Detection	No face detected when image is blank or shows an object	Returns empty bounding box, no crash
UT-03	Embedding Engine	MobileFaceNet produces 128-d vector from valid cropped face	Output shape [128], L2-norm $\approx 1.0 \pm 0.01$
UT-04	Embedding Engine	Same face image run twice produces identical embedding	Cosine similarity = 1.0
UT-05	Cosine Similarity	Genuine pair (same person, two photos) similarity score	Score ≥ 0.75
UT-06	Cosine Similarity	Impostor pair (different persons) similarity score	Score < 0.65
UT-07	Passive Liveness CNN	Live face from camera feed classified correctly	Class = live, confidence ≥ 0.85

UT-08	Passive Liveness CNN	Printed A4 photograph of a face classified as spoof	Class = spoof, confidence ≥ 0.80
UT-09	Active Liveness	Blink detected across 30-frame buffer with genuine eye closure	Blink flag = true within 30 frames
UT-10	Active Liveness	No blink detected when user keeps eyes open for full 30-frame window	Blink flag = false, prompt re-displayed
UT-11	Enrollment (DB write)	User embedding correctly written to encrypted SQLite store	Record retrievable; decrypted vector matches input
UT-12	Purge on ACK	Local log record deleted after simulated server ACK	Count decrements; record ID not found in DB

14.2 Component Integration Testing

Integration tests validate the interaction between modules when wired together inside the native module layer. Tests run on a physical or emulated mid-range device.

Test ID	Integration Scope	Scenario	Pass Criteria
IT-01	Camera → Detection → Embedding	Live camera frame pipeline: capture → detect → crop → embed	Full chain completes < 400 ms; valid 128-d vector returned
IT-02	Embedding → DB → Retrieval	Enroll user; immediately verify with same image	verify() returns authenticated=true, confidence ≥ 0.90
IT-03	Liveness → Recognition Gating	Liveness fails (spoof); recognition must not run	verify() not invoked; response = liveness_fail
IT-04	Enrollment + DB Encryption	Read raw SQLite file off-device; inspect bytes	No plaintext embedding vectors visible (AES-256 confirmed)
IT-05	Sync Service → AWS Endpoint	5 pending logs uploaded when Wi-Fi toggled on	All 5 confirmed in AWS; local DB count = 0 post-purge
IT-06	Sync Retry on Failure	Simulate server 500 error; verify retry logic	Back-off fires at 2s, 4s, 8s, 16s, 32s; record stays in DB
IT-07	JS Bridge → Native Module	Call enroll() from RN JS; receive response	Promise resolves < 500 ms; no bridge exceptions
IT-08	JS Bridge error handling	Call verify() without prior enrollment for userId	Promise rejects with error code USER_NOT_ENROLLED

14.3 System-Level End-to-End Testing

E2E tests simulate real-world field usage across the full authentication flow on a physical device in offline mode (flight mode enabled).

Test ID	Scenario	Steps	Pass Criteria
E2E-01	Successful offline authentication	1. Enroll in Wi-Fi. 2. Enable flight mode. 3. Complete liveness check. 4. Verify face.	Auth granted in < 1 sec; log entry created locally
E2E-02	Spoof rejection — printed photo	1. Enroll user. 2. Print A4 face photo. 3. Hold photo in front of camera. 4. Attempt verify.	Liveness FAIL; auth denied; anti-spoof log entry created
E2E-03	Spoof rejection — screen replay	1. Enroll user. 2. Display face photo on a second device screen. 3. Point camera at screen.	Liveness FAIL; auth denied
E2E-04	Impostor rejection	1. Enroll User A. 2. User B attempts verification.	Cosine similarity < threshold; auth denied
E2E-05	Low-light outdoor simulation	1. Enroll under normal indoor light. 2. Verify in < 50 lux simulated environment.	Auth consistent; latency < 1.5 sec; UI warning displayed
E2E-06	Harsh sunlight simulation	1. Enroll indoors. 2. Verify under 100k+ lux direct light (outdoor test).	Auth succeeds; liveness not degraded; UI shade prompt shown if needed
E2E-07	500-profile scalability	1. Enroll 500 distinct users. 2. Verify each sequentially. 3. Measure latency.	p95 latency < 1 sec; no DB read errors
E2E-08	Offline → online sync cycle	1. Generate 20 offline auth events. 2. Restore network. 3. Wait for auto-sync.	All 20 records in AWS within 60 sec; local DB purged to 0
E2E-09	App crash recovery	1. Create 5 pending log entries. 2. Force-kill app mid-sync. 3. Relaunch app.	No records lost; sync resumes; records not duplicated in AWS

14.4 Device Compatibility Testing

The module must be validated on a representative matrix of physical and emulated devices spanning minimum supported OS versions on both platforms.

Device / Emulator	OS	RAM	SoC	Test Scope
Samsung Galaxy A22	Android 11	4 GB	Helio G80	Full E2E + latency benchmark
Redmi 10 Prime	Android 11	4 GB	Helio G88	Full E2E + liveness testing
Motorola Moto G31	Android 11	4 GB	Helio G85	Enrollment + verification

Android Emulator (API 26)	Android 8.0	3 GB	x86 emulated	Unit + Integration only
iPhone SE (2nd gen)	iOS 13	3 GB	Apple A13	Full E2E + latency benchmark
iPhone 8	iOS 12	2 GB	Apple A11	Basic auth flow + sync
iPhone 12	iOS 14	4 GB	Apple A14	Full E2E + liveness testing
iOS Simulator (iOS 12)	iOS 12	N/A	x86 simulated	Unit + Integration only

14.5 Accuracy & Performance Benchmarking

Benchmarking must be run on a physical mid-range device (3–4 GB RAM). All results must be reported in the Technical Documentation deliverable.

Benchmark	Method	Required Result
Face Recognition Accuracy (TAR@FAR=0.1%)	Evaluate on 500-subject holdout set (genuine + impostor pairs)	> 95% TAR
False Accept Rate (FAR)	1000 different-person impostor pair comparisons	< 0.1%
False Reject Rate (FRR)	1000 same-person genuine pair comparisons	< 5%
Liveness False Accept (live → spoof)	50 live subjects, 5 attempts each	< 2%
Liveness False Reject (spoof → live)	50 printed/screen spoofs tested	< 1%
End-to-End Latency (p50)	100 auth attempts on Snapdragon 665 device	< 800 ms
End-to-End Latency (p95)	100 auth attempts on Snapdragon 665 device	< 1000 ms
Model Inference Latency	50 isolated TFLite inference calls	< 120 ms
Peak Memory Usage at Auth	Android Profiler / Xcode Instruments during auth flow	< 150 MB RAM delta
Total Model Bundle Size	Measure .tflite + .mlmodel assets in release build	≤ 20 MB (target ≤ 12 MB)

14.6 Security & Privacy Testing

Test ID	Test Case	Pass Criteria
SEC-01	No raw facial images stored to disk at any point during enrollment or verification	File system scan post-auth: no .jpg/.png face files in app sandbox

SEC-02	Embeddings are AES-256 encrypted at rest	SQLite DB opened without SQLCipher key returns unreadable bytes
SEC-03	TLS 1.2+ enforced for all AWS sync traffic	Charles Proxy capture shows no plaintext transmission during sync
SEC-04	No biometric data transmitted to any non-AWS endpoint	Network traffic log during full E2E shows zero calls to unlisted domains
SEC-05	Auth logs not accessible via adb backup (Android)	adb backup extract: app data excluded or unreadable (allowBackup=false)
SEC-06	Model files not extractable from APK in usable form	Decompiled APK .tflite asset not directly executable without app runtime

14.7 Testing Tools & Frameworks

Tool	Purpose	License
Jest + React Native Testing Library	Unit and integration tests for RN JS layer	MIT
Detox	E2E automation on Android Emulator and iOS Simulator	MIT
JUnit 5 (Android)	Unit tests for Kotlin native module	Apache 2.0
XCTest (iOS)	Unit tests for Swift native module	Apple (free)
TFLite Benchmark Tool	Measure TFLite inference latency and memory on-device	Apache 2.0
Android Profiler	Memory, CPU, and network profiling during E2E tests	Free (Android Studio)
Xcode Instruments	Memory and CPU profiling on iOS physical device	Free (Xcode)
SQLCipher CLI	Validate DB encryption; attempt open without key	BSD
mitmproxy / Charles Proxy	Verify TLS enforcement and absence of plaintext traffic	Apache 2.0 / Free

14.8 Test Entry & Exit Criteria

Phase	Entry Criterion	Exit Criterion
Unit Tests	Individual modules (face detect, inference, liveness, DB) are code-complete	≥ 90% code coverage; 0 P0 failures
Integration Tests	All native modules compiled; RN bridge wired and building cleanly	All IT-01 to IT-08 pass on one Android + one iOS device
E2E Tests	Integration tests pass; release build APK + IPA generated	All E2E-01 to E2E-09 pass on physical mid-range devices

Benchmarking	E2E tests complete on physical device	All metrics in Section 14.5 meet or exceed targets
Security Tests	Release build generated	All SEC-01 to SEC-06 pass; no critical vulnerabilities open

14.9 Defect Severity Classification

Severity	Definition	Resolution Requirement
P0 — Critical	Auth granted for spoofed face; app crash during auth; any biometric data leak	Must be resolved before submission. No P0 open at exit gate.
P1 — High	Latency > 1.5 sec on target device; accuracy < 93%; sync failing silently	Must be resolved before submission.
P2 — Medium	UI rendering glitch; minor accuracy regression (93–95%); non-critical log field missing	Should be resolved; documented if deferred.
P3 — Low	Cosmetic issues; non-critical documentation gap; minor UX polish items	Deferred acceptable; logged in project backlog.

15. Appendix — Open-Source License Compliance

All technologies specified in this PRD are open-source and require no commercial licensing. The following summary confirms compliance with the hackathon's open-source-only constraint.

Library / Model	License
TensorFlow Lite	Apache 2.0
MobileFaceNet weights	Apache 2.0
MediaPipe	Apache 2.0
ONNX Runtime Mobile	MIT
react-native-vision-camera	MIT
react-native-sqlite-storage	MIT
SQLCIPHER	BSD-style (open-source)
coremltools	BSD 3-Clause
React Native	MIT
AWS SDK (JS)	Apache 2.0

No proprietary SDKs, paid APIs, or licensed model weights are required. All training datasets referenced (MS-Celeb-1M subset, NUAA, CASIA-FASD) are available for research purposes under academic use agreements.